

Les tests unitaires

ce qu'ils vérifient, où ils se trouvent, comment ils tournent

DeepEdge Benchmark · état au 8 juillet 2026

1. Le principe

Les tests unitaires sont des petits programmes qui vérifient automatiquement que le code fait ce qu'il doit. Ici, ils suivent trois règles strictes qui les rendent fiables et rapides : ils sont **hors-ligne** (aucun accès à internet — les vrais cours ne sont jamais téléchargés pendant un test), **déterministes** (mêmes entrées → mêmes résultats, grâce à des graines aléatoires fixées), et fondés sur des **données synthétiques** (une fausse série de prix générée sur place plutôt que de vraies données).

Concrètement, quand un test a besoin d'aller chercher des prix, la fonction de téléchargement est soit remplacée par un faux (« monkeypatch »), soit jamais appelée. Résultat : les tests passent en quelques secondes, partout, sans dépendre du réseau ni des marchés.

2. Cartographie

Les tests sont répartis en quatre zones, selon la partie du projet qu'ils protègent.

Cartographie des tests unitaires

tous hors-ligne · déterministes · aucun accès réseau

models/ *Les 4 modèles de prévision*
test_metrics.py — calcul des métriques (5 tests × 4 modèles)
test_models_common.py — contrat de sortie, pas de fuite du futur, pas de réseau, garde-fous LSTM

model_artifacts/ *Le pipeline entraînement + validation*
test_pipeline.py — Gate 1 (sérialisation), Gate 2 (métriques),
rechargement du modèle = même prévision,
LSTM produit bien ses 5 fichiers

validation/ *La base de suivi*
test_tracking_db.py — 10 tests (idempotence, anti-doublon,
évaluation, agrégations, export CSV)
test_integration.py — flux complet save → evaluate → report

calibration/regime/ *Le moteur de régimes de marché*
test_regime_analytics.py · test_regime_agent.py
test_dashboard_builder.py

3. Les tests des modèles (models/)

test_metrics.py — le calcul des métriques est juste

Vérifie la fonction `compute_metrics()` qui transforme des prévisions en indicateurs. Cinq tests, rejoués sur les 4 modèles :

Test	Ce qu'il garantit
prévision parfaite → erreur nulle	Si la prévision = la réalité, RMSE et MAE valent 0 et la précision directionnelle 100 %.
précision directionnelle (cas connu)	Le % de bons sens de variation est calculé correctement sur un exemple vérifié à la main.
couverture de l'intervalle (pleine)	Si l'intervalle englobe toujours le vrai prix, la couverture vaut 100 %.
couverture de l'intervalle (partielle)	Si un point sort de l'intervalle, la couverture baisse comme attendu.
longueurs incohérentes → erreur claire	Donner des tableaux de tailles différentes lève une erreur explicite, pas un résultat faux.

test_models_common.py — le comportement partagé des 4 modèles

Les 4 modèles (ARIMA, SARIMA, Prophet, LSTM) exposent la même interface. Ces tests, rejoués sur chacun, verrouillent ce comportement commun :

Test	Ce qu'il garantit
aucun accès réseau	La fonction de téléchargement est bien remplaçable par un faux ; le test n'appelle jamais internet.
contrat de sortie respecté	Chaque modèle renvoie un dictionnaire avec les clés attendues (predictions, lower, upper, index, actual, + métriques) et autant de prévisions que de points à prédire, toutes finies.
prévision d'un pas valide	La prévision « prochain pas » renvoie un point et un intervalle finis, avec borne basse $\leq$ point $\leq$ borne haute.
pas de fuite du futur (no look-ahead)	Le test le plus important. On tronque la période de test et on re-prédit : le début des prévisions doit rester identique. Si le modèle « trichait » en regardant des données futures, ce ne serait pas le cas.
garde-fous LSTM	Si on donne au LSTM une série plus courte que sa fenêtre de mémoire (30 points), il lève une erreur claire au lieu de planter obscurément.

4. Les tests du pipeline (model_artifacts/)

test_pipeline.py vérifie la chaîne complète entraînement → validation → sauvegarde, sur données synthétiques :

Test	Ce qu'il garantit
Gate 1 — sérialisation	Le modèle s'entraîne et tous les fichiers attendus sont bien écrits sur le disque.
Gate 2 — métriques	La validation produit un metrics.json avec exactement les bonnes clés, des valeurs finies et cohérentes (largeur min $\leq$ moyenne $\leq$ max), pour les 5 modèles et les 2 horizons.
round-trip de sérialisation	La preuve que le modèle sauvegardé est exploitable : le recharger depuis le disque redonne la même prévision qu'en mémoire. Un artefact vide ou corrompu échouerait ici.
LSTM : les 5 fichiers	Le cas le plus complet — le LSTM produit bien model + scaler + hyperparams + metrics + metadata de bout en bout.

5. Les tests de la base de suivi (validation/)

test_tracking_db.py (10 tests) couvre la base tracking.db : création des tables idempotente, insertion, **anti-doublon** (rejouer un run ne duplique rien), rejet d'un enregistrement incomplet, calcul exact des métriques d'évaluation sur des cas « touché » et « manqué », mise à l'écart des prédictions non échues, agrégations du reporting (dont par régime), et export CSV.

test_integration.py (1 test) vérifie le **flux complet** de bout en bout : enregistrer des prédictions → les évaluer → produire le rapport, avec une base qui s'auto-initialise et un fournisseur de prix simulé.

6. Le rapport de tests dans chaque Run/

Chaque dossier Run/ contient un unit_tests.json : la trace des tests exécutés pour ce modèle au moment du run. On y lit le statut global, le résumé pytest, et la liste détaillée des tests avec leur résultat.

Exemple concret : pour un run ARIMA, unit_tests.json indique « 9 passed, 29 deselected » ; pour un run LSTM, « 11 passed » — deux de plus, car le LSTM a ses propres garde-fous de longueur de série. Le filtre pytest (ex. -k lstm) ne garde que les tests du modèle concerné.

7. La note technique : le deadlock TensorFlow

Point important documenté directement dans le code (models/conftest.py).

Les tests du LSTM ont révélé un piège : si les bibliothèques yfinance/statsmodels sont importées **avant** TensorFlow dans un même processus, le premier entraînement du réseau (.fit()) se bloque indéfiniment — un « deadlock ». Ce n'est pas une lenteur : le processus reste figé pour toujours.

La parade est en place dans conftest.py (le fichier de configuration chargé par pytest avant tout le reste) : **TensorFlow est importé et configuré en premier**, threads limités et GPU désactivé, avant même la collecte des tests. Cela élimine la classe de blocage. Le même correctif d'ordre d'import doit être respecté partout où le vrai LSTM est entraîné.

En résumé

- Tous les tests sont hors-ligne, déterministes, sur données synthétiques.

- Quatre zones : modèles, pipeline, base de suivi, moteur de régimes.
- Points clés protégés : justesse des métriques, contrat des modèles, **pas de fuite du futur**, modèle rechargeable à l'identique, base sans doublon.
- Chaque Run/ garde son unit_tests.json comme preuve.
- Le deadlock TensorFlow est neutralisé en important TF en premier (confest.py).